

Just Draw It: Widgets for Generating and Visualizing Graph Structures in Lean 4

ANONYMOUS AUTHOR(S)

Human reasoning in graph theory is often guided by visual intuition, yet interactive theorem provers like Lean 4 represent graphs through purely textual encodings such as adjacency matrices or graph6 strings. This mismatch steepens the learning curve for formalizing graph theory. To address this gap, we present a suite of widgets and tactics that let users generate, manipulate, and visualize graphs and their substructures on a canvas embedded in the Lean Infoview. All visual data is sent back to the Lean workspace via RPC and automatically parsed into corresponding formal terms, allowing users to directly reference their constructions in the local proof context. We demonstrate that our tools make formal graph theory more accessible through four case studies: (1) counterexamples found via property-based testing, (2) Ramsey graphs, (3) spanning trees, and (4) shortest paths.

CCS Concepts: • **Human-centered computing** → **Visualization toolkits**; • **Mathematics of computing** → **Graph theory**; • **Theory of computation** → **Logic and verification**.

Additional Key Words and Phrases: Interactive Theorem Proving, Lean 4, Graph Theory, User Interfaces

1 Introduction

In the realm of formalized mathematics, combinatorics and graph theory are arguably among the hardest fields to formalize. One key obstacle is the gap between human reasoning and the proof interface. When it comes to graph theory, human mathematicians have always benefited from visual intuition: sketching a graph, tracing a path, or coloring vertices to guide a proof. Most interactive theorem provers (ITPs), however, rely on text-based interfaces, restricting users to inputting and viewing graph terms through textual encodings such as adjacency matrices, edge lists, or graph6 strings. We recognize this pain point within Lean 4 and set out to develop a suite of tools for generating, manipulating, and visualizing graphs and their substructures directly within the Lean Infoview.

Our main contributions can be summarized as follows:

- (1) An interactive widget implemented in JavaScript that allows users to draw simple graphs, weighted simple graphs, directed graphs, and weighted directed graphs, with utilities for subgraph and walk selection, layout formatting and vertices labeling.
- (2) Formalized definitions of `WeightedSimpleGraph` and `WeightedDiGraph`, which are missing from Mathlib, along with a set of parsers that handle the construction of each graph type (as well as substructures such as subgraphs and walks) from adjacency matrices output by the widget.
- (3) A `cex_graph` tactic that visualizes counterexample graphs found by property-based testing in Lean.
- (4) A presenter that allows users to highlight adjacency matrices or graph6 strings in the local context and automatically visualizes them on the canvas in the Infoview.
- (5) Worked examples including Ramsey graphs, spanning trees, and shortest paths, which are common topics in combinatorics and algorithms textbooks such as Brualdi [2] and Cormen et al. [3].

All source code and examples are available at <https://anonymous.4open.science/r/LeanGraphMaker/>.

2 Related Work

2.1 Lean 4

Lean 4 (hereafter called Lean) is an interactive theorem prover and arguably one of the most popular available today. In Lean, proving a proposition is equivalent to constructing a proof term of the corresponding type. Users write proofs using a tactic language, providing a series of steps that guide the construction of this term. Lean’s kernel then verifies that each term is correctly typed. If a term passes this type-checking procedure, the proof is guaranteed to be valid under the rules of Lean’s foundational type theory based on the Calculus of Inductive Constructions.

A key feature of the Lean ecosystem is Mathlib [7], its community-maintained mathematical library. We follow the conventions established in Mathlib to formalize missing common graph types, namely weighted simple graphs and weighted directed graphs. We also utilize theorems and definitions from Mathlib in our worked examples.

2.2 Visual Interfaces for Theorem Proving

Integrating visual interfaces with theorem proving has a long history. The Interactive Derivation Viewer (IDV) [8] provides an interactive interface for navigating proof derivations, highlighting interesting lemmas while hiding intermediate steps. The Incredible Proof Machine [1] takes a more direct approach, allowing users to construct formal proofs by connecting graphical blocks in a port-graph representation, with a focus on pedagogy. More recently, Nawrocki et al. introduced ProofWidgets [6], which provides the widget framework and RPC mechanism that our tools build upon.

ProofWidgets displays widgets in the Lean Infoview panel within VS Code. Widgets are written in TypeScript and can import arbitrary NPM libraries. Nawrocki et al. demonstrate how widgets can leverage powerful libraries such as `three.js`¹ and integrate existing web applications like Penrose [9].² Widgets can be interactive: actions in the widget can invoke corresponding Lean functions via JSON-RPC³ (where the function result is sent back to the widget) or directly induce changes in the Lean proof document. Conversely, widgets can access proof document metadata such as the proof state, making the connection between widgets and the proof document bidirectional.

Since then, other projects have leveraged ProofWidgets for visual reasoning; for example, Sherlock [4] built a graphical rewriting tool for diagrammatic reasoning in monoidal categories in Lean 4. Perhaps most relevant to our work, Narvaez et al. [5] built a primitive widget for visualizing (but not drawing) Lean graph objects, and introduced parsers for decoding graph6 format⁴ into Lean’s SimpleGraph type. We extend this philosophy with a drawing interface that sends the widget’s internal representation back into Lean, as well as dedicated parsers for simple graphs, directed graphs, weighted graphs, and their substructures.

3 Design and Implementation

3.1 Widget User Interface

The graph widget provides a direct-manipulation canvas for constructing and visualizing graphs. The interface supports the following key operations:

- **Vertex creation.** Clicking on empty canvas space places a new vertex. A minimum distance constraint (two vertex radii) prevents overlapping placements.
- **Edge creation.** Selecting two vertices in sequence toggles an edge between them.

¹<https://threejs.org/>

²<https://penrose.cs.cmu.edu/>

³<https://www.jsonrpc.org/>

⁴<https://users.cecs.anu.edu.au/~bdm/data/formats.txt>

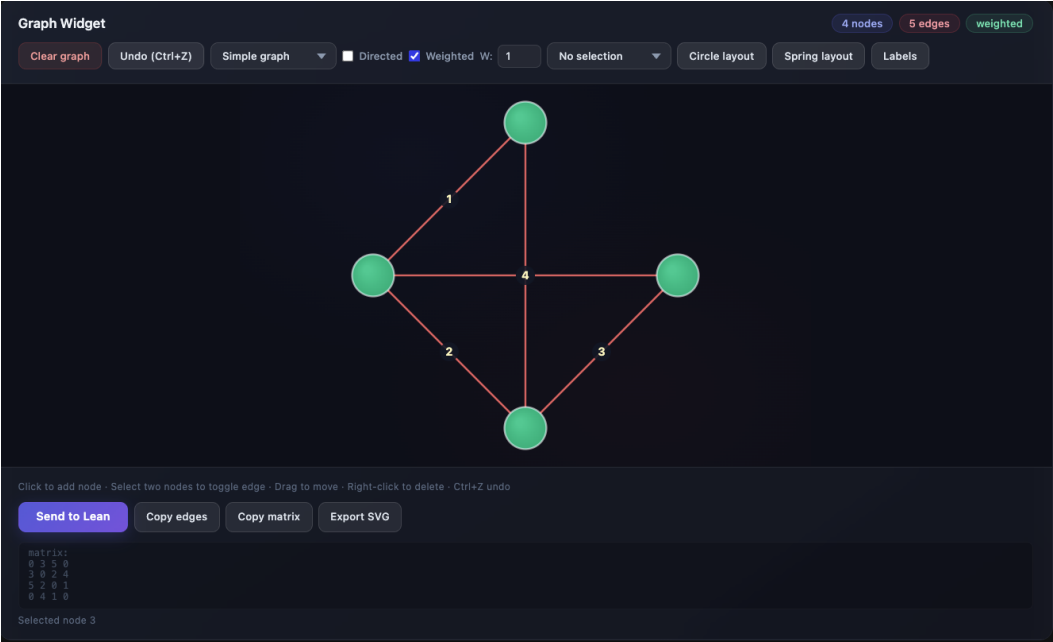


Fig. 1. The graph widget embedded in the Lean Infoview, showing a user-constructed graph with labeled vertices and the control panel.

- **Deletion.** Right-click removes a vertex and its incident edges; the Delete key removes the currently highlighted vertex or edge.
- **Drag repositioning.** Vertices may be dragged to new positions; a collision guard prevents overlap during movement.
- **Layout algorithms.** One-click circle layout and force-directed (Fruchterman–Reingold) spring layout.
- **Zoom.** Mouse wheel scales the canvas; scrollbars appear for panning.
- **Labels.** A toggle button shows or hides vertex index labels.

3.1.1 *Graph Modes.* A dropdown selector determines how edges are interpreted:

- **Simple graph.** Edges are added or removed by toggling (default).
- **Bi-colored graph.** The canvas displays a complete graph where each edge is colored red or blue. Clicking toggles an edge between colors. This mode is designed for Ramsey-theoretic 2-colorings.

Orthogonal checkbox toggles control:

- **Directed.** Edges carry direction; arrowheads are rendered.
- **Weighted.** Edges carry numeric weights displayed as labels at edge midpoints.

3.1.2 *Weight Editing.* Clicking an edge selects it (gold highlight on the weight badge). Digit keys overwrite the current weight on first keystroke, then append on subsequent keystrokes. Backspace removes the trailing digit. If a weight reaches zero upon deselection, the edge is automatically deleted. Toggling “Weighted” on assigns a default weight of 1 to all existing edges.

3.1.3 *Selection Modes.* A second dropdown offers three selection modes:

- **No selection** (default). Standard graph editing.
- **Subgraph selection.** Clicking edges or vertices toggles membership in a highlighted subgraph (gold). Ctrl+Z clears the selection.
- **Walk selection.** Clicking vertices in order builds a directed path (green highlight). Each click extends the walk if an edge exists; clicking the last vertex retracts the final step. Ctrl+Z removes the last vertex from the walk.

3.2 Widget Implementation and Internal Representation

Internally, the widget maintains a nodes array of {id, x, y} objects representing vertex positions, and an edges map keyed by sorted vertex-pair strings ("i-j"), with values {color, weight?, from?, to?}. For subgraph selection, it stores a set of selected edge keys and vertex ids; for walk selection, an ordered array of vertex ids forming the path.

3.2.1 Export and Change Detection. On export ("Send to Lean"), the widget serializes this state into Lean syntax via an RPC call. A change-detection mechanism (hasGraphChanged) compares the current edge state against the initial matrix, accounting for edge direction in directed graphs, to determine whether the parent graph needs re-emission or only the selection binding is required.

3.2.2 Runtime Consistency Check. Before any export, the widget executes a bidirectional consistency check (verifyExport) that verifies the roundtrip between the internal edge map and the generated adjacency matrix:

- (1) Every present edge in the internal map corresponds to a nonzero entry in the constructed matrix (no dropped edges).
- (2) Every nonzero matrix entry corresponds to a present edge in the internal map (no spurious edges).

This assertion guards against state inconsistency: for instance, if a node deletion leaves dangling edge keys, or if the id-to-index mapping becomes stale after vertex reordering. The check runs only when the graph has been modified (i.e., when the matrix will actually be emitted), avoiding unnecessary overhead for pure subgraph or walk selections where the parent graph is unchanged.

3.2.3 Structural Invariants. Additional runtime constraints enforce graph well-formedness:

- **No overlapping vertices.** Node creation is rejected if any existing node lies within the minimum separation distance (two vertex radii), checked against the current state to prevent inconsistencies from rapid successive interactions.
- **No drag overlap.** Vertex repositioning is blocked if the target position would violate the minimum separation from other nodes.
- **Weight consistency.** Edges with weight zero are automatically removed upon deselection, ensuring the exported matrix contains no zero-weight edges that would be semantically invisible.
- **Walk validity.** In walk selection mode, each extension is checked for adjacency before being accepted; the walk is guaranteed to correspond to a valid sequence of edges in the graph.

These client-side invariants complement the kernel-level verification: the widget ensures the exported term is structurally well-formed, while Lean's kernel independently certifies that it satisfies the required mathematical properties via decide.

3.3 Graph Types and Matrix Parsing

3.3.1 *Graph Type Definitions.* We define two graph structures extending Mathlib's SimpleGraph and Digraph:

- `WeightedSimpleGraph V W` extends `SimpleGraph V` with `weight : Sym2 V → W`
- `WeightedDigraph V W` extends `Digraph V` with `weight : V → V → W`

3.3.2 *Matrix Constructors.* Graphs are constructed from adjacency matrices, with each constructor returning the corresponding graph type:

- `Matrix.toSimpleGraph M`: 0/1 symmetric matrix to `SimpleGraph (Fin n)`
- `Matrix.toDigraph M`: nonzero entries define directed edges, returning `Digraph (Fin n)`
- `Matrix.toWeightedSimpleGraph M`: symmetric matrix with zero diagonal; nonzero entries define edges with weights, returning `WeightedSimpleGraph (Fin n) α`
- `Matrix.toWeightedDigraph M`: general matrix; nonzero entries define weighted directed edges, returning `WeightedDigraph (Fin n) α`

Each constructor provides appropriate well-formedness proofs (symmetry, zero diagonal) via `by decide` for concrete matrices.

3.3.3 *Decidability Instances.* We provide `DecidableRel` instances for each constructor so that properties of constructed graphs can be verified by the kernel via `decide`, avoiding the need for `native_decide` (which would introduce Lean's compiler into the trusted base):

- `DecidableRel (M.toSimpleGraph h).Adj`
- `DecidableRel (M.toDigraph).Adj`
- `DecidableRel (M.toWeightedSimpleGraph ..).toSimpleGraph.Adj`
- `DecidableRel (M.toWeightedDigraph).Adj`
- `Decidable (G ≤ H)` for `SimpleGraph` and `Digraph` on `Fintype V`
- `Decidable G.IsTree` via Mathlib's characterization that a graph is a tree if and only if it is connected and $|edgeFinset| + 1 = \text{Fintype.card } V$

3.3.4 *Subgraph Constructors.* The `subgraphOfMatrix` constructors build subgraphs from adjacency matrices:

- `SimpleGraph.subgraphOfMatrix G M`: returns `G.Subgraph` (with $\leq$ proof by `decide`)
- `Digraph.subgraphOfMatrix G M`: returns `{ H // H ≤ G }` (with $\leq$ proof by `decide`)
- `WeightedSimpleGraph.subgraphOfMatrix G M`: returns `G.toSimpleGraph.Subgraph`
- `WeightedDigraph.subgraphOfMatrix G M`: returns `{ H // H ≤ G.toDigraph }`

All are marked `@[reducible]` so that typeclass resolution can unfold them when synthesizing instances for the resulting subgraph.

3.3.5 *Roundtrip Theorems.* We prove that the `matrix → graph → matrix` roundtrip is the identity, ensuring our constructors faithfully preserve adjacency data:

- `Matrix.adjMatrix_toDigraph_eq`
- `Matrix.adjMatrix_toWeightedDigraph_eq`
- `Matrix.adjMatrix_toWeightedSimpleGraph_eq`

3.4 The draw_graph Tactic

The `draw_graph` tactic is invoked with an optional graph identifier:

```
draw_graph          -- empty canvas for interactive construction
draw_graph G        -- pre-populated from local definition G
```

Table 1. Code generation rules for the `draw_graph` tactic.

Modified?	Selection	Output
No	None	(tactic invocation is removed)
No	Subgraph	<code>let G' := G.subgraphOfMatrix !![...]</code>
No	Walk	<code>let p : G.Walk s t := Walk.cons ...</code>
Yes	None	<code>let G := Matrix.toSimpleGraph !![...]</code>
Yes	Subgraph	Graph binding + subgraph binding
Yes	Walk	Graph binding + walk binding

3.4.1 Behavior. The tactic renders the graph widget in the Lean Infoview. When an identifier is provided, the tactic:

- (1) Looks up the identifier in the local context.
- (2) Extracts the adjacency matrix from the expression (handling `Matrix.toSimpleGraph`, `Matrix.toWeightedSimpleGraph`, `Matrix.toDigraph`, `Matrix.toWeightedDigraph`, and `readG6` patterns).
- (3) Reduces via `whnf` if direct extraction fails (supporting definitions that compute a graph via a function).
- (4) Classifies the graph type (directed/weighted) from the expression's type.
- (5) Passes the matrix data, graph type flags, and the identifier name to the widget.

3.4.2 Code Generation. When “Send to Lean” is clicked, the widget's RPC handler generates a source code edit depending on the interaction:

For directed walks, the emitted term uses `Walk.cons (v := k) (by decide)`. For undirected walks, it uses `SimpleGraph.Walk.cons (u := i) (v := j) (by decide)`. Both forms produce kernel-checkable proof terms without relying on `native_decide`.

3.5 Expression Presenter

To support quick visual inspection without invoking a tactic, we provide an expression presenter registered via `@[expr_presenter]`. It activates when the user shift-clicks an expression in the Lean Infoview's selection panel (with `_panel_widgets [ProofWidgets.SelectionPanel]`).

3.5.1 Supported Expressions. The presenter recognizes the following expression types:

- `Matrix V V  $\alpha$` expressions (adjacency matrices)
- String expressions (graph6-encoded strings)
- `SimpleGraph V`, `Digraph V`, `WeightedSimpleGraph V W`, `WeightedDigraph V W` expressions

For graph-typed expressions, the presenter resolves the definition via `whnf` and extracts the underlying matrix or graph6 string.

3.5.2 Rendering. The extracted matrix is passed to the widget in read-only mode. A mode dropdown allows toggling between simple and bi-colored visualization styles. Directed and weighted flags are inferred from the expression's type, ensuring accurate rendering regardless of matrix content.

3.6 The `cex_graph` Tactic

The `cex_graph` tactic is a diagnostic tool that searches for counterexamples to universal statements about graphs via property-based testing, then visualizes the result. It builds on `Plausible`⁵, Lean’s property-based testing library, but requires substantial additional infrastructure to make graph sampling and shrinking feasible.

3.6.1 Sampleable Proxy. `Plausible` cannot directly sample values of type `SimpleGraph (Fin n)`, as graphs are represented as symmetric irreflexive relations without a natural random generator. We introduce `AdjVec n`, a flat boolean array representing the upper triangle of an adjacency matrix, together with:

- An `Arbitrary (AdjVec n)` instance that generates random boolean arrays of the appropriate length.
- A `Shrinkable (AdjVec n)` instance that shrinks by removing one edge at a time (setting individual entries from `true` to `false`), producing minimal counterexamples with the fewest edges.
- A conversion `AdjVec.toSimpleGraph` with a verified symmetry proof.

3.6.2 Mechanism.

- (1) **Goal parsing.** The tactic expects a goal of the form $\forall G : \text{SimpleGraph } (\text{Fin } n), P\ G$. It extracts n (which must be a literal) and the predicate P .
- (2) **Instance stripping.** Any trailing instance binders (e.g., `[DecidableRel G.Adj]`) are automatically synthesized against the proxy graph, enabling the tactic to handle definitions that require typeclass instances as arguments.
- (3) **Proxy construction.** The predicate is lifted to $\forall p : \text{AdjVec } n, P (\text{AdjVec.toSimpleGraph } p)$, and a `Testable` instance is synthesized for the resulting proposition.
- (4) **Testing.** `Plausible` searches for a counterexample among random `AdjVec n` values, with shrinking enabled.
- (5) **Visualization.** If a counterexample is found, the adjacency matrix is extracted from `Plausible`’s output, displayed via the graph widget in the `Infview`, and reported as a `logInfo` message in Lean matrix notation.

Note that `cex_graph` is a diagnostic tactic: it does not produce a kernel-verified proof. When a counterexample is found (or when testing is inconclusive), the goal is discharged via `sorry`. The tactic is intended for exploration and hypothesis testing during proof development, not for closing goals in verified code.

3.6.3 Decidability Requirements. The same restrictions as `Plausible` apply: the property under the quantifier must be decidable. Properties involving `Nonempty`, `Set` membership, or noncomputable definitions (e.g., `SimpleGraph.Colorable`, `SimpleGraph.chromaticNumber`) cannot be tested directly unless the user provides appropriate `Decidable` instances.

```
-- "Every graph on 4 vertices is triangle-free"
-- False: Plausible finds a triangle as a subgraph.
example :
  ∀ G : SimpleGraph (Fin 4),
    ∀ i j k : Fin 4,
      G.Adj i j → G.Adj j k → ¬G.Adj i k := by
  cex_graph
```

⁵<https://github.com/leanprover-community/plausible>

The tactic finds and displays a graph on 4 vertices containing a triangle, refuting the claim that all such graphs are triangle-free.

4 Examples

Due to page constraints, we do not discuss the examples in full detail here. Instead, we encourage interested readers to download the source code and experiment directly. For `Cex.lean`, readers can hover over the `cex_graph` tactic invocation and inspect whether the visualized counterexample matches their intuition. For the remaining files, readers can uncomment the `draw_graph` calls and attempt to construct the correct witness by drawing on the canvas; step-by-step instructions are provided in the comments of each file.

The examples include:

- **Ramsey graphs** (`Ramsey.lean`). Using `draw_graph` to validate lower bounds for small Ramsey numbers by providing explicit graph counterexamples (i.e., 2-colorings of K_n that avoid monochromatic cliques of the required size).
- **Spanning trees** (`SpanningTree.lean`). Finding a spanning tree whose total weight is below a given bound, constructed interactively via subgraph selection on the widget.
- **Shortest paths** (`ShortestPath.lean`). Finding a path between two vertices whose total weight is below a given bound, constructed interactively via walk selection on the widget.

5 Conclusion and Future Work

We have presented a suite of interactive widgets and tactics for generating, manipulating, and visualizing graphs and their substructures within the Lean Infoview. Our tools bridge the gap between visual intuition and formal textual encodings, making formal graph theory more accessible to both students and researchers. Through the `draw_graph` tactic, the expression presenter, and the `cex_graph` diagnostic tactic, users can seamlessly move between graphical interaction and kernel-verified proof terms without relying on `native_decide`.

Our case studies demonstrate that the tools are practical for verifying Ramsey-theoretic lower bounds, constructing spanning trees, and finding shortest paths, all tasks that benefit substantially from visual reasoning.

We plan to develop an interactive Lean game built on the Lean Game Server⁶ that guides students through graph theory exercises using our widgets. Players would progress through levels of increasing difficulty, constructing graph witnesses and verifying properties interactively. We envision this as a pedagogical tool for introductory courses in combinatorics and algorithms, lowering the barrier to entry for both formal verification and graph theory. More broadly, we hope our tools inspire further development of visual interfaces for other areas of formalized mathematics within Lean.

References

- [1] Joachim Breitner. 2016. Visual Theorem Proving with the Incredible Proof Machine. In *Interactive Theorem Proving (ITP 2016) (Lecture Notes in Computer Science, Vol. 9807)*. Springer, 123–139. doi:10.1007/978-3-319-43144-4_8
- [2] Richard A. Brualdi. 2010. *Introductory Combinatorics* (5th ed.). Pearson.

⁶<https://adam.math.hhu.de/>

[3] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. 2022. *Introduction to Algorithms* (4th ed.). MIT Press.

[4] Sam Ezeh. 2024. Graphical Rewriting for Diagrammatic Reasoning in Monoidal Categories in Lean4. In *15th International Conference on Interactive Theorem Proving (ITP 2024) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 309)*, Yves Bertot, Temur Kutsia, and Michael Norrish (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 41:1–41:8. doi:10.4230/LIPIcs.ITP.2024.41

[5] David E. Narváez, Cruise Song, and Ningxin Zhang. 2024. Formalizing Finite Ramsey Theory in Lean 4. In *Intelligent Computer Mathematics: 17th International Conference, CICM 2024, Montréal, QC, Canada, August 5–9, 2024, Proceedings* (Montreal, QC, Canada). Springer-Verlag, Berlin, Heidelberg, 91–108. doi:10.1007/978-3-031-66997-2_6

[6] Wojciech Nawrocki, Edward W. Ayers, and Gabriel Ebner. 2023. An Extensible User Interface for Lean 4. In *14th International Conference on Interactive Theorem Proving (ITP 2023) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 268)*, Adam Naumowicz and René Thiemann (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 24:1–24:20. doi:10.4230/LIPIcs.ITP.2023.24

[7] The mathlib Community. 2020. The Lean Mathematical Library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*. ACM, New Orleans, LA, USA. doi:10.1145/3372885.3373824

[8] Steven Trac, Yury Puzis, and Geoff Sutcliffe. 2007. An Interactive Derivation Viewer. In *Proceedings of the Second International Conference on Computer Science: Theory and Applications (CSR) (Lecture Notes in Computer Science, Vol. 4649)*. Springer, 305–316. doi:10.1007/978-3-540-74510-5_31

[9] Katherine Ye, Wode Ni, Max Krieger, Dor Ma’ayan, Jenna Wise, Jonathan Aldrich, Joshua Sunshine, and Keenan Crane. 2020. Penrose: from mathematical notation to beautiful diagrams. *ACM Trans. Graph.* 39, 4, Article 144 (Aug. 2020), 16 pages. doi:10.1145/3386569.3392375